DSC 395T Algorithms and Data Structures	Individual Project
Flux Equilibrium Clustering	Programming Assignment #5

In this assignment, you will **work individually** to implement the FEC clustering algorithm. In addition to the Python standard library and Pytest, you will be allowed to use NumPy, as one goal of this project is to give you experience with NumPy. You should use NumPy version 2.2.6 with Python 3.10.14.

1 Overview

Clustering algorithms accept as input a set of points in some n-dimensional space and produce as output a set of clusters, so that all points within a cluster are closely related by some metric. For example, a k-means clustering would place the points into one of k clusters, such that all of the points in a given cluster are geometrically closer to each other than to any point in any other cluster.

In this assignment, you will implement Flux Equilibrium Clustering (FEC), a density-seeking graph-based clustering algorithm. Each point in the dataset is treated as a source of flux (e.g., water), which flows downhill on a *k-nearest-neighbor (k-NN) graph* toward the global centroid. As the algorithm iterations, the flux redistributes until an equilibrium is reached. Points that accumulate as sinks become the centroids of clusters.

The input is an $n \times m$ array, where n is the number of points and m is the dimension of the space. The output is an $n \times 1$ array where each element is the cluster number for that point, so the number of unique values is the number of clusters.

2 Parameter Definitions

The following definitions will be useful.

- k: Number of nearest neighbors for graph construction (default: 10)
- σ: Bandwidth parameter for Gaussian kernel weights (default: $0.2\times$ median pairwise distance)
- α: Fraction of stored flux redistributed per iteration (default: 0.5)
- T: Number of transport iterations (default: 25)
- ϵ: Outgoing flux threshold for sink detection (default: 1^{-4})
- β: Reinforcement parameter for flux-aware weights (default: None, uses $\log(1 + f)$)
- $d(i)$: Distance from point x_i to global centroid c, i.e., $d(i) = \|x_i - c\|$, where $\|v\|$ is the Euclidean norm of a vector v.
- c: Global centroid of all data points, $c = \frac{1}{n}\sum_{i=1}^{n} x_i$

3 Algorithm Steps

Step 1. Pre-compute:

- Compute the global centroid $c = \frac{1}{n}\sum_{i=1}^{n} x_i$.
- Calculate the distance from each point to the centroid: $d(i) = \|x_i - c\|$.
- If σ is not specified as a program input, set $\sigma = 0.2 \times \text{median}(\{\|x_i - x_j\| : i < j\})$.
- Set the effective value of k: $k_{\text{eff}} = \min(k, n - 1)$ neighbors.
- Build a symmetric k-NN graph with edge weights using exponential:

$$w_{ij} = \exp\left(-\frac{\|x_i - x_j\|^2}{\sigma^2}\right)$$

Step 2. Initialize Flux:

- Set initial stored flux $f_i(0) = 1$ for each node i.

Step 3. Simulate flow:

- For T iterations, each node i sends flux to neighbors j that satisfy the following relaxed downhill criterion:

$$j \text{ is downhill from } i \text{ if } d(j) \leq d(i) + \tau$$

  where $\tau = 0.1 \times \sigma$ is a tolerance parameter.
- Compute flux-aware edge weights (for $t > 0$):

$$\tilde{w}_{ij}(t) = w_{ij} \cdot \phi(f_j(t))$$

$$\text{where } \phi(f) = \begin{cases} 1 & \text{if } t = 0 \\ \log(1+f) & \text{if } t > 0 \text{ and } \beta = \text{None} \\ f^{\beta} & \text{if } t > 0 \text{ and } \beta \neq 0 \end{cases}$$

- Send flux along each edge:

$$\delta_{ij}(t) = \alpha \cdot \tilde{w}_{ij}(t) \cdot \frac{f_i(t)}{\sum_{m:d(m)\leq d(i)+\tau} \tilde{w}_{im}(t)}$$

- Update flux:

$$f_i(t+1) = f_i(t) - \sum_j \delta_{ij}(t) + \sum_h \delta_{hi}(t)$$

Step 4. Detect Sinks:

- A vertex i is a sink if its outgoing flux in the final iteration satisfies the following constraint:

$$\sum_j \delta_{ij}(T) < \epsilon$$

- If no sinks are detected, designate the point closest to the centroid as a sink.

Step 5. Assign Clusters with Cycle Detection:

- For each sink s, assign cluster label $\ell(s)$.
- For each non-sink node i, determine the steepest downhill neighbor:

$$\text{steepest}(i) = argmax_{j:d(j)\leq d(i)+\tau} \delta_{ij}(T)$$

- Trace the steepest path to a sink with cycle detection:
  - Follow path $i \to \text{steepest}(i) \to \text{steepest}(\text{steepest}(i)) \to \ldots$
  - If a cycle (closed path) is detected, break it by designating the current node as a new sink
  - Assign all nodes in the path the same cluster label as the terminal sink

Step 6. Post-Processing: Small Cluster Merging:

- Define the minimum cluster size: $s_{\min} = \max(2, \lfloor n/20 \rfloor)$
- For each cluster C with $|C| < s_{\min}$:
  - Compute cluster centroid: $\mu_C = \frac{1}{|C|} \sum_{x \in C} x$
  - Find nearest larger cluster: $C^* = argmin_{C':|C'|\geq s_{\min}} \|\mu_C - \mu_{C'}\|$
  - Merge C into C^*

4 Your Assignment

You will implement the following methods in the module **fec.py**:

```
class FluxEquilibriumClustering:
    def __init__(self, ...) -> None:
    def fit(self, X: np.ndarray) -> "FluxEquilibriumClustering":
```

The `__init__` method stores parameters as instance attributes.

The `fit` method computes the cluster labels following the steps above.

Acknowledgments. We thank Omkar Bajaj for creating this assignment. And we thank Saffron Liu, Joseph Zhu, and Pablo Esparza for their helpful comments on this assignment.